Pravadh DocSign — Security Review: Scanner Findings & Justifications

Scanner: Salesforce Code Analyzer v5 (PMD, SFGE, ESLint, RetireJS, Regex, CPD) with rule selectors `AppExchange`, `Security`, `Recommended`. Result (25 Sep 2026): `Recommended:Security` selector **0 violations**; `AppExchange` selector **28 findings, all Moderate (severity 3) false positives**, justified below (22 `ProtectSensitiveData`, 5 `AvoidHardcodedCredentialsInVarAssign`, 1 `AvoidUnauthorizedApiSessionIdInVisualforce`). Reports: `scan/DocSign-AppExchange.html`, `scan/DocSign-Security.html`.

1. System-mode data access (explicitly declared)

The following classes run **without sharing** and use `WITH SYSTEM_MODE` / `AccessLevel.SYSTEM_MODE` deliberately. None are `global`, `@AuraEnabled`, `@RemoteAction` or REST-exposed; each is reachable only after authorization by one of: (a) a user-mode DML/SOQL gate on the same record, (b) a verified signer token, or (c) a scheduled/platform-event process.

Class	Why system mode is required	Authorization before entry
`DocSignSigningService` (called by the `with sharing` page controllers `DocSignSigningController`, `DocSignPageImageController`)	External signers use a Salesforce Site as the guest user, who owns no records and has no object permissions (by design; secure guest sharing).	256-bit random token (only SHA-256 hash stored), optional salted access code with 5-attempt lockout, envelope status/expiry/routing-order checks on every request. Signer can only touch their own recipient row and fields of their own envelope. All values validated server-side; Name/Email/Date are system-populated.
`DocSignAuditService`, `DocSignAuditGuard`	Audit events are written for signers and system processes; users only have Read on audit events (immutability).	Called internally only. Inserts blocked unless the service opens the guard; update/delete always blocked by trigger.
`DocSignRoutingService`, `DocSignLifecycle`, `DocSignEvents`, `DocSignReminderSender`	Deliver signing links, void links, publish events on behalf of the envelope, including after an external signer acts.	Invoked after a user-mode update of the same envelope (cancel/resend/send), a verified signer action, or the maintenance job.
`DocSignCompletionService`, `DocSignCertificateBuilder`, `DocSignEventHandler`, `DocSignEventJob`	Finalization (signed PDF, certificate, Files, optional record update) runs asynchronously as the platform-event process user after the last signature.	Triggered only by the package's own platform event; idempotent (`Finalized__c`), re-verifies every recipient has signed.

2. `AvoidGlobalModifier` (6)

`DocSignGenerateDocumentAction`, `DocSignSendForSignatureAction`, `DocSignEnvelopeActions`, `DocSignCancelEnvelopeAction`, `DocSignResendEnvelopeAction`, `DocSignRecipientInput` are **Flow invocable actions / Apex-defined types**. They must be `global` so subscriber Flows can use them. They run `with sharing` and delegate to user-mode services.

3. `ProtectSensitiveData` (22) and `AvoidHardcodedCredentialsInVarAssign` (5) — false positives

The rules match names containing *sign*, *hash*, *key* or *token* (e.g. `Signing_Order__c`, `Signature_Method__c`, `Document_Hash__c`, `Envelope_Key__c`, the local variable `hash` in `DocSignPdfPrepController`, `needsSignature`). None hold secrets or credentials, and no value assigned in code is a credential (the flagged assignments are a computed SHA-256 digest, picklist values such as `Typed` / `Drawn`, and a Boolean):

- `Token_Hash__c`, `Access_Code_Hash__c`: **one-way SHA-256 hashes**; raw tokens/codes are never stored (tokens are emailed once, access codes shared out-of-band).
- `*_Hash__c`: document/audit integrity digests (public by nature; printed on the Certificate of Completion).
- `Envelope_Key__c`: a random UUID used as a public envelope reference (like a DocuSign Envelope ID); it grants no access.
- `Signing_Site_Url__c`: a public Site URL.
- `Signature_Image__c`, `Signature_File_Id__c`: the signer's visual mark (and the Salesforce File that stores it for PDF rendering). Evidence displayed on the signed PDF, not a credential.
- `Signature_Method__c`, `Signed_Date__c`, `Failed_Access_Attempts__c`, `Signing_Order__c`: picklist, timestamp, counter and ordering values.
- `Certificate_Version_Id__c`, `Signed_Version_Id__c`: Salesforce ContentVersion ids of the generated PDFs; access is still governed by Files sharing.

4. `@lwc/lwc/no-async-operation` (3)

`docSignPromo` uses `setTimeout` / `setInterval` to show a dismissible product card and re-show it 15 minutes after dismissal; timers are cleared in `disconnectedCallback`. Admins can disable the card in DocSign Settings.

5. Complexity / style (Cyclomatic, Cognitive, NcssCount, ExcessiveParameterList, etc.)

Maintainability guidance, not security. The merge-engine parser and signing validation are intentionally explicit.

6. Uploaded-PDF signing (PDF.js, remoting, page images)

Area	Control
Third-party library	`DocSignPdfEngine` static resource bundles **PDF.js 4.10.38** (Apache-2.0), which includes the fix for **CVE-2024-4367**. `getDocument` is called with `isEvalSupported: false` and `enableXfa: false`; embedded PDF JavaScript is never executed. Source and rebuild steps: `docs/docsign/pdf-engine/`.
Where PDF parsing runs	Only on the internal `DocSignPdfPrep` Visualforce page, for authenticated users with the DocSign permission sets. **The public signing site never parses PDFs**: signers receive server-rendered page images only.
Upload validation	FileType/extension must be PDF, ≤ 50 MB, `%PDF-` signature verified server-side and client-side, ≤ 100 pages and ≤ 600 fields, password-protected PDFs rejected.
Page images	`savePage` accepts only `` payloads beginning with the JPEG signature, ≤ 1.2 MB, stored as ContentVersion (user mode) on the DocSign Document. `saveLayout` verifies every referenced image belongs to that document (`FirstPublishLocationId`).

Area	Control
Layout validation	Page numbers, coordinates (must lie within the page), field types (allow-list), roles (≤ 20, renumbered), label/value lengths are validated server-side; preparer values are HTML-escaped into the document.
Edit authorization	`DocSignPdfPrepController` is `with sharing`, all SOQL `WITH USER_MODE`, DML `as user`; `UserRecordAccess.HasEditAccess` is checked for the current user (that query is explicitly `WITH SYSTEM_MODE` because `UserRecordAccess` is itself the access check). Documents that were sent are locked.
Signer page images	The `DocSignPageImage` page (`DocSignPageImageController` → `DocSignSigningService.pageImage`, guest, read-only, no DML) re-validates the signing token, envelope state/expiry/routing, the version-id format, and that the image is registered on that envelope's document before returning it (one page per call).
Cross-frame messaging	The LWC embeds the editor with a one-time 128-bit nonce and its own origin. The editor posts only to that origin, which the controller validates against Salesforce domains. The LWC accepts messages only from Salesforce origins and with the matching nonce. Because Visualforce remoting is unavailable on the header-less editor page, the editor asks the LWC to call its Apex methods: the LWC relays only an allow-list of five `@AuraEnabled` methods (`getContext`, `savePage`, `saveLayout`, `getPdfData`, `saveDraft`) of the `with sharing` controller, always for the document id it opened, so the relay grants nothing the user could not already call.
URL parameters	`id`, `o`, `n` on `DocSignPdfPrep` are pattern-validated (Id type, https Salesforce origin, alphanumeric nonce) and HTML-escaped before use.
Test data	`AvoidHardcodingId` in `DocSignPdfPrepTest` is a deliberately non-existent ContentVersion id used to prove foreign images are refused.

Other controls reviewers should know

- **CRUD/FLS:** all user-facing SOQL uses `WITH USER_MODE`; DML uses `insert/update/upsert/delete as user`; merge paths are validated with `isAccessible()` per relationship hop; dynamic SOQL uses only describe-validated names and bind variables (no string concatenation of user input).
- **XSS:** all merged values are HTML/XML-escaped (`raw` filter is opt-in for admin-authored rich text). The signing page renders the document inside a sandboxed iframe without `allow-scripts`. The Template preview is a server-rendered PDF (no `innerHTML`). Only `data:image/png|jpeg;base64` signature images are accepted.
- **Signing link leakage:** `<meta name="referrer" content="no-referrer">`, `robots noindex`, tokens rotated on resend/reminder and nulled on completion/decline/cancel/expiry.
- **CSRF:** Visualforce form postbacks (built-in view-state CSRF protection). No GET side effects: pages have no load actions and constructors are read-only.
- **No callouts / no external scripts.** Only outbound links (promo card) open pravadhlabs.com domains in a new tab with `rel="noopener noreferrer"`.
- **No sensitive data in logs:** no `System.debug`.
- **Immutability:** audit events cannot be edited/deleted; envelopes that were sent cannot be deleted; SHA-256 hash chain verified on every workspace load and on the Certificate of Completion.
- **Recommended Site settings for customers:** HTTPS only, Clickjack protection = "Allow framing by the same origin only", assign only the *Pravadh DocSign Signing Guest* permission set to the guest user.

Session ID in DocSignPdfPrep.page (`$Api.Session_ID`) — `AvoidUnauthorizedApiSessionIdInVisualforce` (1)

The PDF preparation editor (authenticated Salesforce users only, never the public Site) downloads the uploaded PDF with `GET /services/data/v67.0/sobjects/ContentVersion/{id}/VersionData` on the page's own Visualforce domain. The browser cannot read the file through `/sfc/servlet.shepherd/...` because that redirects to the `*.file.force.com` content domain and is blocked by CORS. The session ID is used only in the `Authorization` header of that same-origin request to the org's own REST API; it is never placed in a URL, stored, logged, or sent to any other host, and the request honours the user's own file access. If the user lacks API access the editor falls back to an Apex read (files up to 2 MB) or asks the user to pick the same file locally (verified by size).

Blob.toPdf images referenced by `/sfc/servlet.shepherd/version/download/068...`

Server-rendered PDFs reference committed Salesforce Files versions by relative URL (the PDF service does not render `data:` URIs). Only IDs matching `068` + 12/15 alphanumerics are accepted (`DocSignRenderService.isFileUrl`), never external URLs.

Signing page: no side effects on page load

`DocSignSign.page` has no `action` attribute and its constructor is read-only (Visualforce also forbids DML there). The "Document Viewed" event (recipient/envelope status, Viewed date, audit entry) is recorded by `recordView()`, called once through an `apex:actionFunction` postback after the page has rendered, so it is covered by the Visualforce view-state CSRF protection. It still requires the signing token and, when set, a verified access code. Side benefit: email link scanners that only fetch the URL do not mark a document as viewed. The brand colour is validated server-side as a hex colour and set with `JSENCODE` from a script, not merged into the `<style>` block.

Checkmarx Source Code Scanner (Partner Security Portal): findings and justifications

Latest scan: 27 Sep 2026 (scan started 21:06, Partner Security Portal), preset `PortalAll`, CxEngine 9.7: **56 findings, 0 Critical** (was 83 before the hardening below). CRUD Violation 7, FLS Create 23, FLS Update 11, Sharing 8, CSRF In Aura LWC 5, SOQL/SOSL User Mode Missing 2. Every one is listed and justified below.

0 Critical. No SOQL/SOSL injection, Reflected/Stored XSS, Client DOM XSS or code injection, URL redirection, XSRF, CRUD Delete or Lightning API version findings. The flagged items below are by design and are explained per query. Before this scan was repeated for the submitted version, the package was also hardened (see *Hardening done in response to this scan*).

A. FLS Create (23) and FLS Update (11): system-mode DML

Every flagged statement uses an explicit `AccessLevel.SYSTEM_MODE`. All user-initiated DML in the package uses `as user` / `AccessLevel.USER_MODE`. System mode is used only where the running user is not, and must not be, the one granting the write:

Flagged location	Who runs it	Why system mode	Authorization before the write
`DocSignAuditService` L65-86 (audit rows: 14 FLS Create paths)	Any DocSign process	Audit events are **read-only for every user** by design (permission sets grant Read only). They are the tamper-evident evidence trail, so users must never be able to create or edit them directly.	Called only by package code after the action being recorded was authorized. `DocSignAuditGuard` blocks inserts outside the service and all updates/deletes.
`DocSignSigningService` L94-99, 148-191, 236-238 (recipient view status, signed field values, voiding other tokens)	Salesforce Site **guest user** (external signer)	The guest user owns no records and has **no object or field permissions** (secure guest sharing); signers are not Salesforce users.	256-bit token (stored only as a SHA-256 hash), optional salted access code with 5-attempt lockout, envelope status/expiry/routing-order checks on every request. Writes are limited to the token holder's own recipient and field records. Values are validated server-side; Name/Email/Date values are set by the system.
`DocSignRoutingService` L38-49, `DocSignLifecycle` L65-67 (token hashes, sent dates, statuses)	Sender (after a user-mode envelope operation), maintenance job	Signing status, token hashes and dates are **evidence fields that users are deliberately not allowed to edit** (read-only in the package permission sets). Only the routing engine may set them.	Invoked right after the sender's user-mode insert/update of the same envelope (send, resend, cancel, reminder) or by the scheduled job.
`DocSignCompletionService` L157 + L208-211 (signer image Files), L163-171 (recipient file ids), L182-196 `saveFinalFile` (signed PDF, Certificate of Completion, their file links)	Platform-event subscriber (Automated Process) or the viewing user via `resumePending`	Finalization runs after the last *external* signer; there is no user context with rights to the signer's records.	Triggered only by the package's own platform event. Re-verifies that every recipient has signed. Idempotent (`Finalized__c`).

B. Sharing (8): `without sharing` classes

Class	Justification
`DocSignSigningService` (called by the public `DocSignSign` / `DocSignPageImage` page controllers, which are `with sharing` and do no SOQL/DML)	Used by the Site guest user, who can own no records; see the token authorization above. No DML or data exposure happens before the token is validated. `DocSignPageImage` returns one page image and only if it belongs to the token holder's envelope.
`DocSignRoutingService`, `DocSignLifecycle`, `DocSignReminderSender`, `DocSignEvents`	Run after the sender's user-mode operation, after a verified signer action, or from the scheduled maintenance job. They must reach every recipient of the envelope (including after an external signer acts) and publish the package's own platform event.

Class	Justification
`DocSignCompletionService`, `DocSignCertificateBuilder`	Asynchronous finalization as the platform-event process user (see A).
`DocSignAuditService`	Writes audit events, which are read-only for every user (see A). Called only by package code after the action being recorded was authorized.

None of these classes is `global`, `@AuraEnabled`, `@RemoteAction` or REST-exposed. The only entry points a user can call are `with sharing` controllers and invocable actions.

C. Apex CRUD Violation (7): `Messaging.sendEmail`

All seven paths end at `DocSignEmailService.send` → `Messaging.sendEmail(msgs, false)`. Every message is built with `setSaveAsActivity(false)` and no `setTargetObjectId` / `setWhatId`, so **sending creates no Salesforce records** (no Task or EmailMessage). There is no object for a CRUD check. Recipients come from DocSign recipient records the user created in user mode, or, for Settings → *Send test email*, an address typed by an admin (custom-permission gated).

D. Apex CSRF In Aura LWC (5)

All five paths start in LWC **event handlers triggered by the user**, never in `connectedCallback` or `renderedCallback`: `docSignRecordPanel.handleUploadFinished` (after the user uploads a PDF), `docSignTemplateManager.handleRowAction` (Clone menu item), `.save` (Save button), `.runPreview` (Preview button), `docSignSettings.save` (Save button). `@AuraEnabled` calls go through the Lightning framework, which adds its own CSRF token to every request. The Apex methods run `with sharing` with `as user` DML, and Settings changes need the admin custom permission.

E. Apex SOQL SOSL User Mode Missing (2)

Path	Justification
`DocSignPageImageController` → `DocSignSigningService.pageImage` L404 (`ContentVersion ... WITH SYSTEM_MODE`)	Guest signer. Token, envelope state and routing are validated first. The version id must match the `068` pattern **and** be one of the page images registered on that envelope's document.
`DocSignSigningTest` L39	Test class path, not shipped logic.

Hardening done in response to this scan

Second pass (28 Sep 2026): 83 findings reduced to 56, confirmed by the rescan above.

- **DocSign Settings save is now user mode.** `DocSignHomeController.saveSettings` checks create/update access on the `DocSign_Settings__c` object and every field it writes, then uses `upsert as user`. The Admin permission set grants access to the custom setting (`customSettingAccesses`). Clears 10 FLS Create and 10 FLS Update findings.
- **Template activate, delete and Word-file upload use Lightning Data Service** (`updateRecord` / `deleteRecord`) in `docSignTemplateManager`; the Apex methods `setActive`, `deleteTemplate` and `setTemplateFile` were removed. Clears 3 CSRF findings.

- **Public page controllers are `with sharing`.** `DocSignSigningController` and `DocSignPageImageController` do no SOQL or DML themselves and only call the token-checked `DocSignSigningService`. Clears 2 Sharing findings.
- **`DocSignFileService` is `with sharing` and user mode only** (document generation). Saving the signed PDF and the Certificate of Completion moved into `DocSignCompletionService.saveFinalFile`, next to the rest of finalization. Clears 1 Sharing finding.
- **`DocSignCompletionService.resumePending` queries `WITH USER_MODE`**, so a user can only trigger the retry for envelopes they can read. Clears 1 User Mode Missing finding.

First pass:

- **Evidence fields are now read-only for all DocSign users** (both permission sets and page layouts). Users can't set or change signing status, token or access-code attempt data, sent/viewed/signed/declined dates, decline reasons, signature images and method, IP address or user agent, consent date, signer field values, completion dates, final or audit hash pointers, or signed/certificate file ids. Only the package's signing, routing and completion services write them, which is why those services use system mode (A). Recipient status now comes from the picklist default (`Created`) on insert.
- The signing page no longer performs any DML on GET (no page `action`). The view is recorded by a CSRF-protected `apex:actionFunction` postback.